TECHNICAL ASSIGNMENT

Full-Stack Developer (Mid-Level)

Support Desk — Java Spring Boot API + Separated Web Frontend

1. Purpose

This assignment shows how you design, structure, secure, test and deliver a complete feature across the full stack. We are not looking for a finished product. We are looking for clean, readable, well-reasoned code, correct handling of business rules, and sound engineering judgement.

It is pitched at **mid-level** engineers. Beyond working CRUD, we expect to see layered architecture, authentication and authorization, consistent error handling, database migrations, automated tests, and a clear explanation of your trade-offs.

2. The Scenario

Build an internal tool called **Support Desk**, used by a support team to track customer issues from the moment they are reported until they are closed.

The interesting part is not the CRUD. It is the **workflow rules** and the **security model**. A large part of the evaluation is whether rules are enforced correctly and in the right place in your code.

3. Mandatory Technical Requirements

Area	Requirement
Backend	**Java Spring Boot** (Java 17+ and Spring Boot 3.x recommended). Maven or Gradle.
Database	Relational (**PostgreSQL** or MySQL) is recommended because of the relationships and transactional rules. **MongoDB** is allowed, but you must justify the choice in the README and still enforce all rules and uniqueness constraints.
API style	RESTful JSON API with correct HTTP verbs, status codes and resource naming.
Frontend	**Separated from the backend** (own folder or repository, own build, talks to the API over HTTP only). Use **React, Vue or Angular**. Plain HTML/CSS/JavaScript is accepted but must still be structured into modules.
Authentication	**User login and register** with JWT (see section 7).
Version control	**Git** with a clear, meaningful commit history. The repository must be **public on GitHub**.
Runnable	A reviewer must be able to start the whole system locally by following the README. Docker Compose is strongly recommended.

4. Data Model

Implement at least the following entities. You may add fields or tables where it improves the design; explain additions in the README.

Entity	Fields
User	Id · Email (unique) · Password hash · Role (Admin / Agent) · Enabled · Created date Register and login operate on this entity. Passwords are never stored or returned in plain text.
Agent	Id · User (one-to-one) · Full name · Email (unique) · Department (Technical / Billing / General) · Active (true/false)
Ticket	Id · Reference (auto-generated, unique, human-readable, e.g. `TCK-2026-0001`) · Title · Description · Customer name · Customer email · Priority (Low / Normal / High / Critical) · Status (New / In Progress / Resolved / Closed) · Assigned agent (optional, a new ticket starts unassigned) · Created date · Last modified date · Resolved date · Closed date · Due date (**calculated by the system, never sent by the client**) · Reopen count · Resolution summary · Version (for optimistic locking) · Deleted flag (soft delete)
Comment	Id · Ticket · Author (user) · Body · Visibility (Public / Internal) · Created date
Attachment	Id · Ticket · Original file name · Stored file name · Content type · Size · Uploaded by · Uploaded date
TicketHistory	Id · Ticket · Actor (user) · Action (Created / Updated / StatusChanged / Assigned / Unassigned / PriorityChanged / Commented / Attached / Deleted) · Old value · New value · Timestamp Append-only audit trail.

Relationships: a ticket has many comments, attachments and history entries. An agent can be assigned many tickets. A user has at most one agent profile.

5. Business Rules

These must be enforced on the **backend**, in the domain/service layer. The frontend may also reflect them for a better user experience, but the API must never rely on the client to do so.

5.1 Core rules

R1 **Due date** is derived from the priority at creation time: Critical → 4 hours after creation, High → 1 day, Normal → 3 days, Low → 7 days. If the priority is changed while the ticket is still open (New or In Progress), the due date is recalculated from the **original creation date**.

R2 **Allowed status transitions:** New → In Progress, In Progress → Resolved, Resolved → Closed, and Resolved → In Progress (reopening). Any other transition is rejected with a clear error. A ticket cannot skip from New to Resolved, and a Closed ticket can never be reopened.

R3 A ticket cannot move to **In Progress** unless an **active** agent is assigned to it.

R4 An **inactive** agent cannot be assigned to any ticket.

R5 A **Closed** ticket is **read-only**: no edits, no assignment changes, no status changes, no new comments, no new attachments.

R6 **Resolved date** and **closed date** are set by the system when the corresponding transition happens, never by the client.

R7 A ticket is **overdue** when its due date has passed and its status is neither Resolved nor Closed.

5.2 Extended rules (mid-level)

R8 **Reference generation** must be unique and safe under concurrent requests. The sequence resets per year (`TCK-<year>-<4+ digit counter>`). Explain how you prevent duplicates.

R9 **Resolving requires a resolution summary.** Moving to Resolved without a non-empty summary is rejected. The summary is stored on the ticket.

R10 **Reopening requires a reason** (stored as a comment and in history) and increments the reopen count. A ticket may be reopened at most **3 times**; the fourth attempt is rejected.

R11 A ticket that is **In Progress cannot be unassigned**. It must be reassigned directly to another active agent, or moved back through a valid transition first.

R12 An agent **cannot be deactivated** while they hold tickets in New or In Progress. The API responds with the list of blocking ticket references.

R13 **Auto-close:** Resolved tickets with no activity for a configurable number of days (default 7) are closed automatically by a scheduled job. Closed date is set by the system and a history entry is written with the system as actor.

R14 **Priority changes** are only allowed while the ticket is New or In Progress. Changing priority on Resolved or Closed tickets is rejected.

R15 **Deletion** is a soft delete, allowed only for Admins and only for tickets in New status with no comments. Deleted tickets never appear in lists or searches.

R16 **Optimistic concurrency:** updates and status changes carry the ticket version. If the version is stale the API returns `409 Conflict` instead of silently overwriting.

R17 **Every state-changing operation** on a ticket writes a TicketHistory entry in the same transaction as the change.

R18 **Internal comments** are visible to agents and admins only. Public comments are visible to everyone with access to the ticket.

6. Roles and Permissions

Capability	Admin	Agent
Register / login	Yes	Yes
View ticket list and details	All tickets	All tickets
Create and edit tickets	Yes	Yes
Change status	Any valid transition	Only on tickets assigned to them
Assign / unassign agents	Yes	Self-assign only (unassigned tickets)
Delete tickets (R15)	Yes	No
Create / activate / deactivate agents	Yes	No
View dashboard statistics	Yes	Yes

Authorization is enforced on the server (method security or equivalent). Hiding a button in the UI is not authorization.

7. Authentication Requirements

- **Register:** `POST /api/auth/register` with email, password and full name. Validate password strength. New self-registered users get the Agent role and an Agent profile. A seeded Admin account exists for first use.
- **Login:** `POST /api/auth/login` returns a short-lived **JWT access token** and a **refresh token**.
- **Refresh / logout:** `POST /api/auth/refresh` issues a new access token; logout invalidates the refresh token.
- Passwords hashed with **BCrypt** (or Argon2). Never log or return them.
- All endpoints except register, login, refresh and health require a valid token. Unauthenticated requests return `401`; insufficient role returns `403`.
- Secrets (JWT signing key, DB credentials) come from environment variables or config, never hard-coded in the repository.

8. Backend — Spring Boot API

8.1 Architecture

- Layered structure (controller → service → repository) with business rules in the **service/domain layer**, not in controllers or repositories. Package by feature or by layer, but be consistent and explain it.
- Spring Data JPA (or Spring Data MongoDB) for persistence, **Flyway or Liquibase** for schema migrations. No `ddl-auto=update` in the delivered configuration.
- **DTOs** for all requests and responses (MapStruct or manual mapping). Never expose JPA entities through the API.
- Bean Validation on request DTOs plus domain-level checks for rules the annotations cannot express.

8.2 Endpoints

Method and path	Purpose
`POST /api/auth/register` · `/login` · `/refresh` · `/logout`	Authentication (section 7)
`GET /api/tickets`	Paginated list. Query params: `page`, `size`, `sort`, `search` (reference, title, customer name), `status`, `priority`, `assignedAgentId`, `overdueOnly`
`GET /api/tickets/{id}`	Ticket with comments (visibility-filtered), attachments metadata and computed `overdue` flag
`POST /api/tickets`	Create ticket (status New, unassigned, due date computed)
`PUT /api/tickets/{id}`	Update editable fields only (title, description, customer fields, priority). Requires version
`DELETE /api/tickets/{id}`	Soft delete (R15)
`PUT /api/tickets/{id}/assignee` · `DELETE /api/tickets/{id}/assignee`	Assign or unassign an agent

Method and path	Purpose
`POST /api/tickets/{id}/status`	Dedicated status-change endpoint. Body: target status, optional resolution summary / reopen reason, version. **Explain in the README why a dedicated endpoint is preferable to a generic update.**
`GET /api/tickets/{id}/allowed-transitions`	Returns the transitions currently allowed for this ticket and user, so the UI never duplicates the rule table
`POST /api/tickets/{id}/comments`	Add a comment (public or internal)
`POST /api/tickets/{id}/attachments` · `GET /api/attachments/{id}`	Upload and download files (section 8.4)
`GET /api/tickets/{id}/history`	Audit trail, newest first
`GET /api/agents` · `GET /api/agents/{id}`	List (with search by name or email, optional department and active filters) and get by id
`POST /api/agents` · `PUT /api/agents/{id}` · `PATCH /api/agents/{id}/active`	Admin only. Activate / deactivate enforces R12
`GET /api/dashboard/summary`	Counts by status and priority, overdue count, average resolution time, tickets per agent

8.3 Error handling

- A single global exception handler (`@RestControllerAdvice`) returns **RFC 7807 Problem Details** (or an equivalent consistent JSON shape): `type`, `title`, `status`, `detail`, a stable **machine-readable `code`** (for example `INVALID_STATUS_TRANSITION`, `AGENT_INACTIVE`, `TICKET_CLOSED`, `VERSION_CONFLICT`), a `traceId`, and field-level `errors` for validation failures.
- A rejected status transition states **why** (current status, requested status, which rule was broken, and what is allowed instead).
- Correct codes: `400` validation, `401`, `403`, `404`, `409` conflict or rule violation, `422` where you choose to use it. Document your convention.
- Clients never receive a raw stack trace or internal class name.

8.4 File upload

- Allow attachments on tickets (images, PDF, text). Enforce a size limit and a content-type allow-list **on the server**.
- Store files outside the web root with generated names. Never trust the client-provided file name (path traversal).
- Attachments are blocked on Closed tickets (R5).

8.5 Logging and observability

- Structured application logging with SLF4J/Logback. Log level and format configurable per profile.
- A **correlation / trace id** per request, included in logs and in error responses.
- Log business events (status changes, assignments, auth failures) at appropriate levels. Never log passwords, tokens or full personal data.
- Expose a health endpoint (Spring Actuator).

8.6 Data and configuration

- Migrations plus a **seed dataset**: roughly 5 agents (including 1 inactive), 1 admin, and about 25 tickets spread across all statuses and priorities, including a few overdue ones, some with comments, and at least one reopened ticket.
- Configuration per profile (`dev`, `test`), CORS restricted to the frontend origin, OpenAPI/Swagger UI available in dev.
- Database indexes on the columns used for filtering and search, with a short justification.

9. Frontend — Separated Web App

Use React, Vue or Angular (or structured vanilla JS). The frontend only talks to the backend through HTTP. All HTTP calls live in a **service / API layer**, never directly inside UI components.

9.1 Authentication screens

- Register and login pages with client-side validation and readable server errors.
- Token handling through a central HTTP interceptor/client: attaches the token, refreshes it once on `401`, and logs the user out cleanly if refresh fails.
- Route guards: unauthenticated users are redirected to login; admin-only screens are hidden and protected.

9.2 Ticket list page

- Table or card list showing reference, title, customer, priority, status, assigned agent and due date.
- Overdue tickets visually highlighted.
- Search input (**debounced**) and filters for status, priority, agent and overdue-only, all wired to the API. **Server-side** filtering, sorting and pagination only.
- Filters, sort and page are reflected in the **URL query string**, so refresh and back/forward keep the state.
- Loading, empty and error states.

9.3 Ticket detail page

- Full ticket information, comment thread (with a public/internal toggle and a form to add a comment), attachments list with upload and download, and a **history timeline**.
- Status action that offers only the transitions returned by the allowed-transitions endpoint, prompting for the resolution summary or reopen reason where required.
- Agent assignment control (inactive agents not offered).
- Everything disabled once the ticket is Closed.
- Handles a `409` version conflict gracefully (tell the user the ticket changed and offer to reload).

9.4 Create / edit ticket

- Form with client-side validation and clear messages. Server-side validation and rule violations are mapped back onto the right fields or shown in a readable summary.

9.5 Other screens

- **Dashboard** using the summary endpoint: counts by status, overdue count, simple chart or cards.
- **Agents page** (admin): list, search, create, activate/deactivate with the R12 error shown clearly.

- Delete with a confirmation step.

9.6 UX and quality

- **Responsive, clean UI** that works on a phone-width viewport.
- Basic accessibility: labelled form fields, keyboard-reachable controls, sufficient colour contrast, focus states.
- Reasonable state management for the chosen framework (for example server-state caching and a small client store). Explain your choices in the README.

10. Quality Requirements

10.1 Automated tests

Area	Minimum	Must cover
Backend unit tests	**8 meaningful tests**	Status transition matrix (valid and invalid, parameterised), due date calculation per priority and recalculation on priority change, assignment rules (R3, R4, R11), resolution and reopen rules (R9, R10)
Backend API / security tests	**3 tests**	At least one controller or integration test (MockMvc or Testcontainers) covering `401`, `403` and a rejected transition returning the structured error body
Frontend tests	**4 tests**	A service (API calls, token handling) and a component (for example the status-action component or the list filters). Jest, Vitest or Jasmine/Karma

10.2 Git and delivery

- Clear commit history: small, focused commits with meaningful messages (Conventional Commits recommended). Feature branches are welcome. A single "initial commit" containing everything is not acceptable.
- `.gitignore` is correct. No secrets, build output or uploaded files committed.
- `docker-compose.yml` that starts the database, backend and frontend (strongly recommended), plus an `.env.example`.

10.3 README.md

The README must cover:

- How to run the backend, frontend and database locally (including seeded login credentials).
- Architecture overview and **where you placed the business rules and why**.
- Why a dedicated status endpoint, and your error-response convention.
- Database choice, indexing, and how you handled reference-number concurrency (R8).
- Security decisions: token lifetimes, refresh strategy, password policy, file-upload safeguards.
- Assumptions you made where the specification was ambiguous.
- What you would improve or add with more time.
- Roughly how long the assignment took you.

11. Bonus Features (Optional)

- Role-aware email or in-app notifications when a ticket is assigned or becomes overdue.
- Business-hours-aware due date calculation (configurable working hours and holidays).
- Real-time updates (WebSocket or Server-Sent Events) on the ticket detail page.
- CSV export of the filtered ticket list.
- Rate limiting on login and register, and account lockout after repeated failures.
- CI pipeline (GitHub Actions) that builds and runs both test suites on every push.
- Dark mode, i18n, or any other creative feature you think adds value.

12. How We Evaluate

Criteria	Weight	What we look for
Business rules correctness	25%	All rules enforced server-side, in the right layer, with edge cases handled
Backend design and code quality	20%	Layering, DTOs, transactions, migrations, naming, readability
Security	15%	JWT flow, password hashing, role enforcement, upload safety, no secrets in the repo
Frontend functionality and UX	15%	Server-side list behaviour, state handling, error surfacing, responsiveness
Testing	10%	Meaningful assertions on rules, not just coverage numbers
API and error design	5%	Consistent, machine-readable errors; sensible status codes
Git history and documentation	10%	Commit quality, README clarity, honest trade-off discussion

A complete, well-tested core with honest notes about what is missing scores higher than a sprawling, half-working submission. Prioritise sections 3 to 10 before attempting section 11.

13. Submission

- Push your code to a **public GitHub repository** (backend and frontend in one repo or two linked repos).
- Email the **GitHub URL**, the completed **employment form** and your **resume** to **bill.hong@neurogine.com**.
- Make sure the repository README lets a reviewer run everything without contacting you.